

Lexical Analyzer

CHAPTER 03

Definition

The main task of lexical analysis is to read input characters in the code and produce tokens.

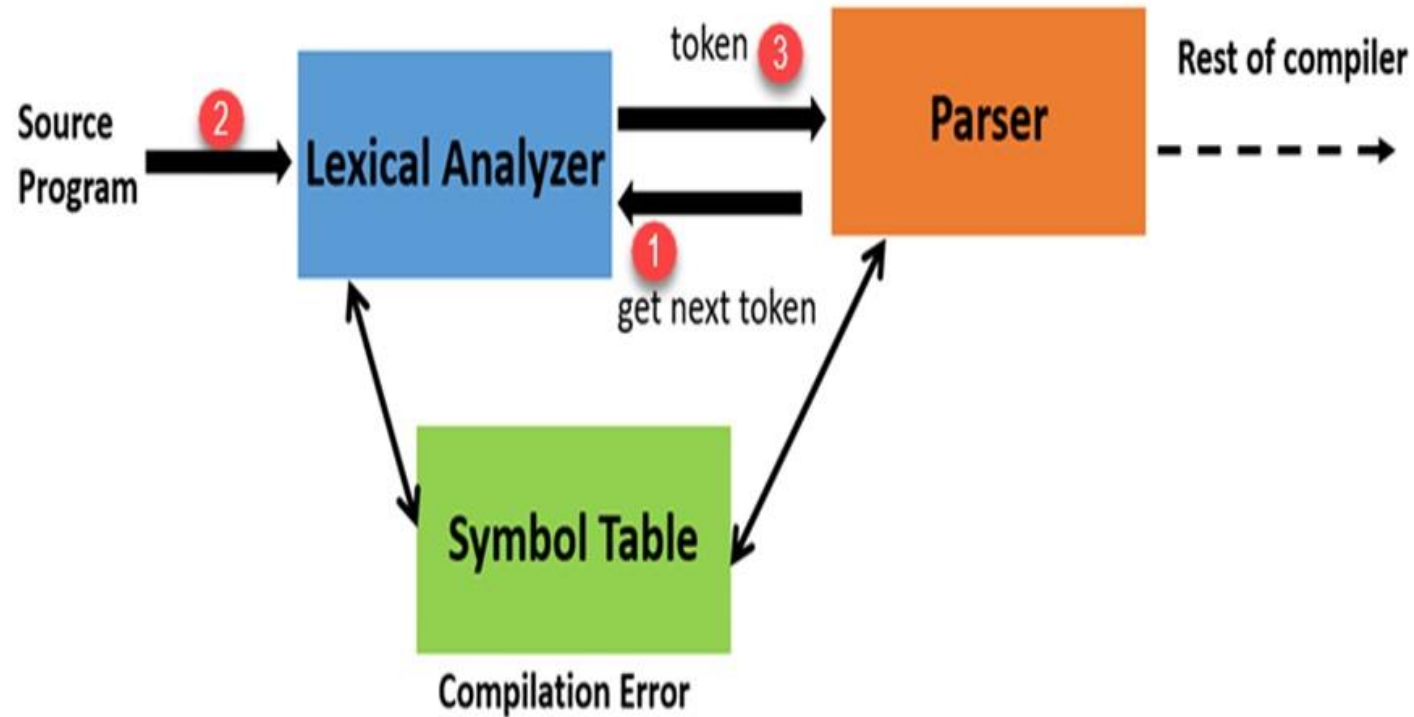
Lexical analyzer scans the entire source code of the program. It identifies each token one by one. Scanners are usually implemented to produce tokens only when requested by a parser.

How lexical Analyzer Works?

Here is how recognition of tokens in compiler design works-

1. “Get next token” is a command which is sent from the parser to the lexical analyzer.
2. On receiving this command, the lexical analyzer scans the input until it finds the next token.
3. It returns the token to Parser.

How lexical Analyzer Works?



Lexeme, Token, Pattern

What's a lexeme?

A lexeme is a sequence of characters that are included in the source program according to the matching pattern of a token. It is nothing but an instance of a token.

What's a token?

Tokens in compiler design are the sequence of characters which represents a unit of information in the source program.

What is Pattern?

A pattern is a description which is used by the token. In the case of a keyword which uses as a token, the pattern is a sequence of characters.

Roles of the Lexical analyzer

- Helps to identify token into the symbol table
- Removes white spaces and comments from the source program
- Correlates error messages with the source program
- Helps you to expands the macros if it is found in the source program
- Read input characters from the source program

Example of Lexical Analysis, Tokens, Non-Tokens

Consider the following

```
#include <stdio.h>

int maximum(int x, int y) {
    // This will compare 2 numbers
    if (x > y)
        return x;
    else {
        return y;
    }
}
```

Examples of Tokens created

Lexeme	Token
int	Keyword
maximum	Identifier
(	Operator
int	Keyword
x	Identifier
,	Operator
int	Keyword
Y	Identifier
)	Operator
{	Operator
If	Keyword

Examples of nontokens

Type	Examples
Comment	// This will compare 2 numbers
Pre-processor directive	#include <stdio.h>
Pre-processor directive	#define NUMS 8,9
Macro	NUMS
Whitespace	/n /b /t

Lexical Errors

A character sequence which is not possible to scan into any valid token is a lexical error.
Important facts about the lexical error:

- Lexical errors are not very common, but it should be managed by a scanner
- Misspelling of identifiers, operators, keyword are considered as lexical errors
- Generally, a lexical error is caused by the appearance of some illegal character, mostly at the beginning of a token.

Error Recovery in Lexical Analyzer

Here, are a few most common error recovery techniques:

- Removes one character from the remaining input
- In the panic mode, the successive characters are always ignored until we reach a well-formed token
- By inserting the missing character into the remaining input
- Replace a character with another character
- Transpose two serial characters

Lexical Analyzer vs. Parser

Lexical Analyser	Parser
Scan Input program	Perform syntax analysis
Identify Tokens	Create an abstract representation of the code
Insert tokens into Symbol Table	Update symbol table entries
It generates lexical errors	It generates a parse tree of the source code

Why separate Lexical and Parser?

- The simplicity of design: It eases the process of lexical analysis and the syntax analysis by eliminating unwanted tokens
- To improve compiler efficiency: Helps you to improve compiler efficiency
- Specialization: specialized techniques can be applied to improve the lexical analysis process
- Portability: only the scanner requires to communicate with the outside world
- Higher portability: input-device-specific peculiarities restricted to the lexer

Home Work

1. Identify the lexemes and their corresponding tokens in the following statement: `printf (“Simple Interest=%f\n”, si)`
2. Give the tokens, lexemes, and patterns for the following statement:

`If (t>=y)`

`{`

`t=y -3; y=t+1;`

`printf(“%d%d”,t,y);`

`}`